Product Requirements Document (PRD)
Simple Grocery Tracker

Version: 1.0
Last Updated: November 3, 2025
Document Owner: Product Team
Status: Draft

📋 Table of Contents

---

1. Executive Summary

1.1 Product Overview
Simple Grocery Tracker is a mobile application that helps users track their grocery spending in real-time while shopping. By scanning products and monitoring their budget, users can make informed purchasing decisions and avoid overspending.

1.2 Problem Statement
- Shoppers often exceed their grocery budget without realizing it until checkout
- Comparing prices across stores is difficult without historical data
- Manual tracking on paper or calculators is tedious and error-prone
- No easy way to review past shopping trips and spending patterns

1.3 Solution
A streamlined mobile app that enables users to:
- Scan products via barcode while shopping
- Track spending against a preset budget in real-time

- View shopping history and analytics
- Compare prices across different stores over time

1.4 Target Platform
- **Primary:** React Native (iOS & Android)
- **Backend:** Supabase (PostgreSQL, Auth, Storage)
- **Camera:** Expo Camera for barcode scanning

---

2. Product Vision & Objectives

2.1 Vision Statement
"Empower every shopper to stay within budget and make smarter purchasing decisions through effortless real-time tracking."

2.2 Primary Objectives
1. **Budget Control:** Help users stay within their grocery budget 90% of the time
2. **Ease of Use:** Enable product scanning and addition in under 10 seconds
3. **Price Awareness:** Provide historical price data to identify savings opportunities
4. **User Adoption:** Achieve 60% retention rate after 30 days

2.3 Success Criteria
- 10,000+ downloads in first 3 months
- Average session duration: 15+ minutes
- 70% of users complete at least one checkout session
- 4.0+ star rating on app stores

---

3. User Personas

3.1 Primary Persona: Budget-Conscious Parent
Name: Sarah, 34
Occupation: Marketing Manager, Mother of 2
Tech Savviness: Medium

Goals:
- Stay within monthly grocery budget of $600
- Track spending across weekly shopping trips
- Compare prices between Walmart and Target
- Plan meals based on actual spending patterns

Pain Points:

- Always overspends by $50-100/month
- Can't remember prices from previous trips
- Kids add items to cart without asking
- Checkout total is always a surprise

User Story:
"As a budget-conscious parent, I want to scan items as I shop so that I can see my running total and stay within my weekly $150 budget."

3.2 Secondary Persona: Frugal College Student
Name: Marcus, 21
Occupation: College Student, Part-time Barista
Tech Savviness: High

Goals:
- Maximize limited grocery budget ($100/month)
- Find cheapest options for staple items
- Track spending to avoid using credit card
- Share budgeting wins with roommates

Pain Points:
- Very tight budget with no room for error
- Multiple stores but no time to price compare
- Often has to put items back at checkout
- Doesn't want to create yet another account

User Story:
"As a college student, I want to use the app without creating an account so that I can quickly track my cart total without any friction."

3.3 Tertiary Persona: Deal Hunter
Name: Patricia, 58
Occupation: Retired Teacher
Tech Savviness: Low-Medium

Goals:
- Find the best deals and discounts
- Track prices across multiple stores
- Build shopping lists based on price history
- Save money for travel and hobbies

Pain Points:
- Forgets which store had better prices
- No easy way to track price changes over time
- Overwhelmed by too many features in other apps
- Wants simple, large buttons and text

User Story:
"As a deal hunter, I want to see price history for each product so that I know when I'm getting a good deal."

---

4. User Journey & Flow

4.1 High-Level User Flow
```
[Login/Guest] → [Start Session] → [Scan Products] → [View Cart] → [Complete/Cancel] → [History/Analytics]
```

4.2 Detailed User Journey Map

Journey Stage 1: Onboarding
Touchpoint: App Launch → Login Screen

User Action	System Response	User Emotion
Opens app for first time	Shows splash screen → Login options	Curious
Sees "Continue as Guest"	Displays prominent guest button	Relieved
Taps Continue as Guest	Navigates to Home tab	Confident

Exit Criteria: User reaches Home screen

Journey Stage 2: Session Setup
Touchpoint: Home Tab → Start Session Form

User Action	System Response	User Emotion
Taps "Start Grocery Session"	Shows session form modal	Focused
Enters session name (e.g., "Weekly Shop")	Accepts input	Neutral
Enters store name (e.g., "Walmart")	Accepts input	Neutral

Sets budget ($150)	Accepts input, validates number	Optimistic
Selects grocery type (Weekly)	Shows dropdown options	Organized
Taps "Start Session"	Creates session, navigates to Scan tab	Excited

Exit Criteria: Active session created, ready to scan

Journey Stage 3: Product Scanning
Touchpoint: Scan Tab → Camera View

User Action	System Response	User Emotion
Camera opens automatically	Shows viewfinder with scan area	Ready
Scans barcode	Beep sound, captures barcode	Satisfied
Takes product photo	Camera switches to photo mode	Engaged
Enters brand name	Text input field	Neutral
Enters product name	Text input field	Neutral
Enters price ($5.99)	Numeric input, formatted as currency	Aware
Sets quantity (2)	Stepper control	Satisfied
Taps "Add to Cart"	Shows success message, clears form	Accomplished
Repeats for 10-15 items	Running total visible in tab badge	In control

Exit Criteria: All desired products added to cart

Journey Stage 4: Cart Review
Touchpoint: Cart Tab → Cart View

User Action	System Response	User Emotion
Taps Cart tab	Shows all cart items with images	Reviewing
Sees total: $138.45 of $150	Shows budget bar (92% filled, green)	Relieved
Notices duplicate item	Swipes left on item	Correcting
Taps Delete	Removes item, updates total to $130.45	In control
Updates quantity of milk (1→2)	Total updates to $135.44	Satisfied
Reviews final list	All items displayed clearly	Confident

Exit Criteria: Cart review complete, ready for checkout

Journey Stage 5: Checkout

Touchpoint: Cart Tab → Checkout Action

User Action	System Response	User Emotion
Taps "Complete Checkout"	Shows confirmation dialog	Decisive
Confirms action	Shows loading spinner	Anticipating
System saves checkout	Success message + confetti animation	Accomplished
Navigates to History tab	Shows new checkout record	Proud

Alternative Path: User taps "End Session" → Cart cleared, no history saved → Neutral emotion

Exit Criteria: Shopping trip saved to history (or cancelled)

Journey Stage 6: Post-Trip Review
Touchpoint: History Tab / Analytics Tab

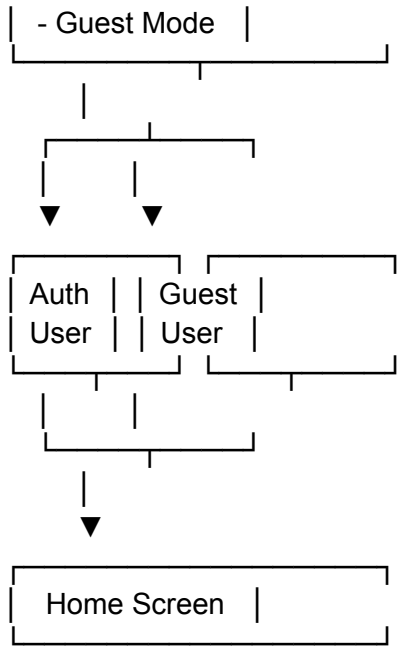
User Action	System Response	User Emotion
Opens History tab	Shows list of past checkouts	Reflective
Taps on last week's trip	Shows detailed breakdown	Analytical
Compares two trips	Notices spending down 12%	Proud
Opens Analytics tab	Shows charts and trends	Informed
Sees monthly spending: $580	Under $600 budget goal	Accomplished

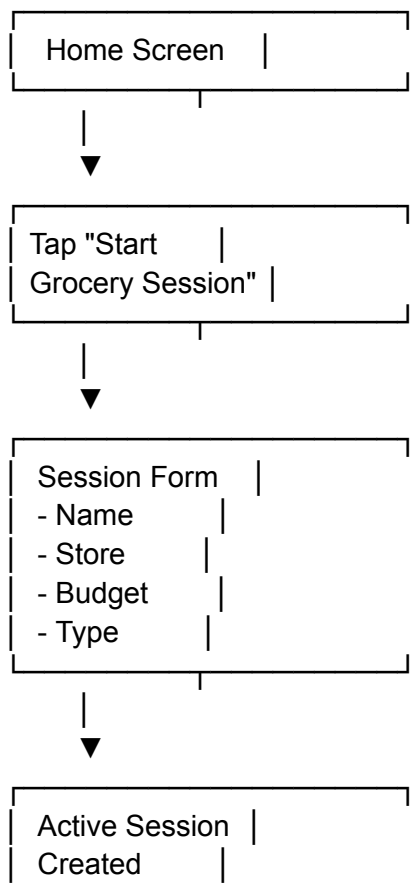
Exit Criteria: User closes app feeling informed and in control

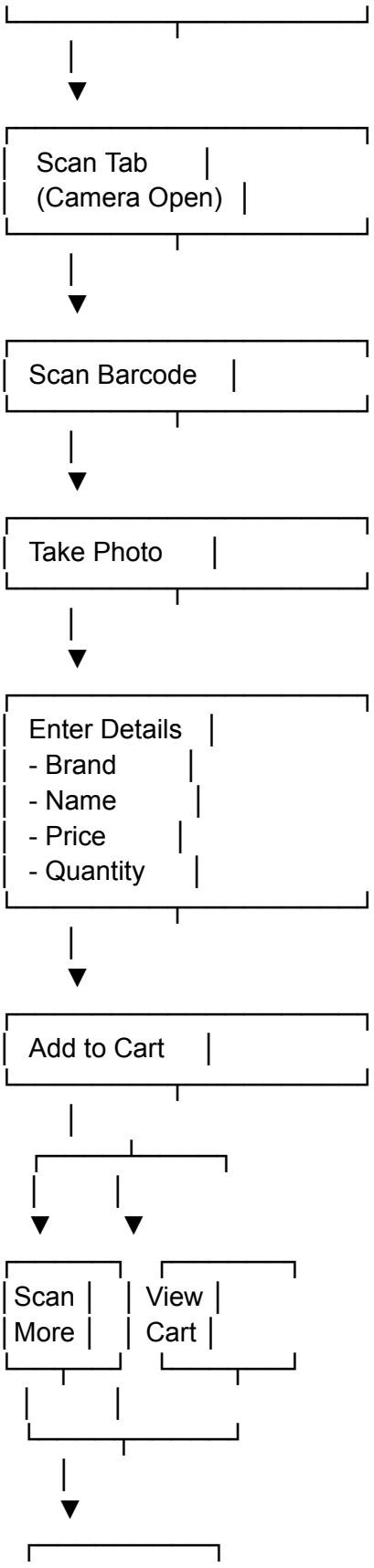
---

4.3 User Flow Diagrams

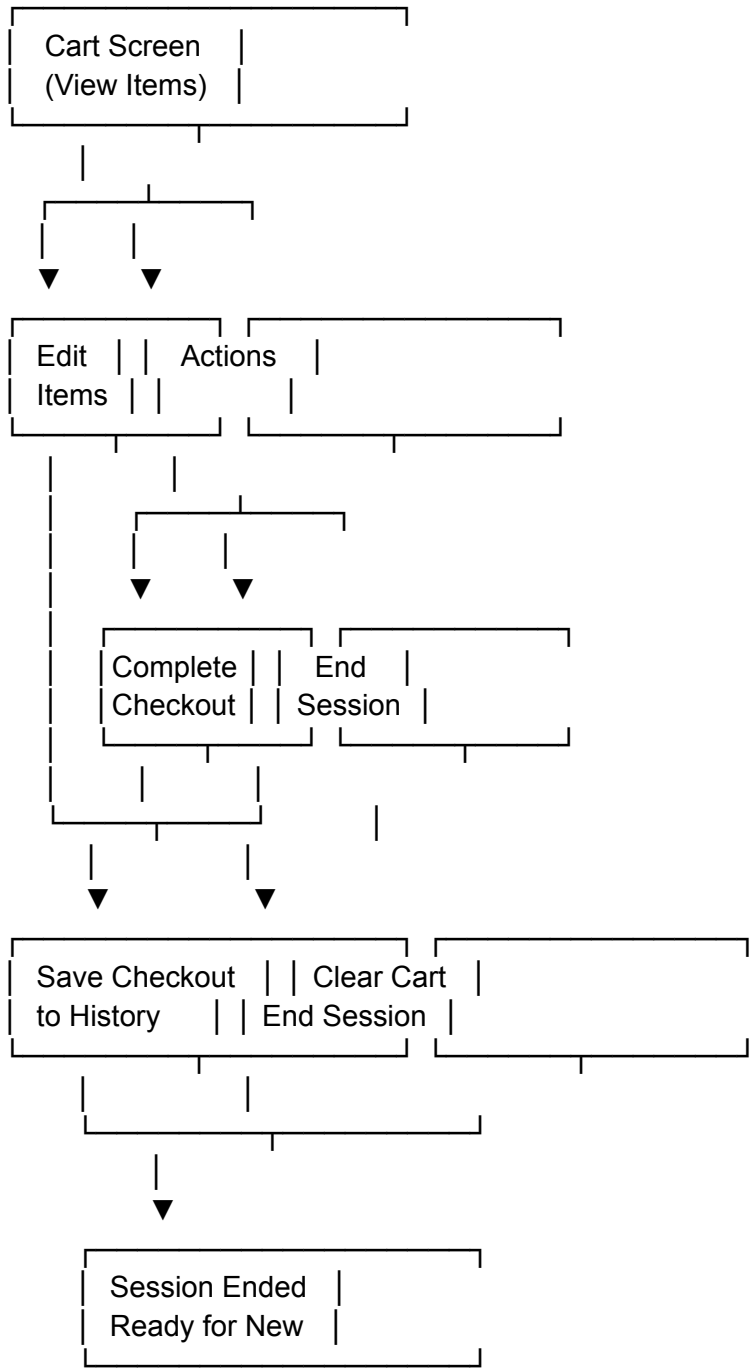
Authentication Flow
```
┌─────────────────────┐
│  App Launch         │
└──────────┬──────────┘
           │
           ▼
┌─────────────────────┐
│  Login Screen       │
│  - Sign In          │
│  - Sign Up          │
```




```

#### **Session & Scanning Flow**

```



Scan Tab
(Camera Open)
Scan Barcode
Take Photo
Enter Details
- Brand
- Name
- Price
- Quantity
Add to Cart
Scan More
View Cart

```
  │Continue │
  └─────────┘
```

Checkout Flow
```


```

---

5. Functional Requirements

5.1 Authentication & User Management

FR-1.1: User Registration
Priority: P0 (Must Have)
User Story: *As a new user, I want to create an account so that my shopping history is saved across devices.*

Acceptance Criteria:
- [ ] User can register with email and password
- [ ] Password must be minimum 8 characters
- [ ] Email validation prevents invalid addresses
- [ ] User receives email confirmation after registration
- [ ] Error messages display for duplicate emails
- [ ] Registration completes in < 3 seconds

Technical Notes:
- Use Supabase Auth for user management
- Hash passwords with bcrypt
- Store user profile in `profiles` table

FR-1.2: Guest Mode
Priority: P0 (Must Have)
User Story: *As a privacy-conscious user, I want to use the app without creating an account so that I can try it first.*

Acceptance Criteria:
- [ ] "Continue as Guest" button visible on login screen
- [ ] Guest users can access all core features (scan, cart, session)
- [ ] Guest data stored only in device AsyncStorage
- [ ] Guest users cannot access History or Analytics tabs
- [ ] Prompt to create account shown after first checkout
- [ ] Guest data can be migrated to account upon registration

Technical Notes:
- Use AsyncStorage for guest data persistence
- Implement data migration flow from guest → authenticated
- Clear AsyncStorage on logout

FR-1.3: Login & Logout
Priority: P0 (Must Have)
User Story: *As a returning user, I want to log in quickly so that I can access my saved data.*

Acceptance Criteria:
- [ ] User can log in with email and password
- [ ] "Remember Me" option keeps user logged in for 30 days
- [ ] Error messages for incorrect credentials
- [ ] Password reset via email option
- [ ] Logout button in Settings/Profile
- [ ] Session persists across app restarts

5.2 Grocery Session Management

FR-2.1: Start Grocery Session
Priority: P0 (Must Have)
User Story: *As a shopper, I want to start a new grocery session so that I can track my spending for this specific trip.*

Acceptance Criteria:
- [ ] "Start Grocery Session" button on Home screen
- [ ] Form includes: Session Name, Store Name, Budget, Grocery Type
- [ ] Session Name: Text input (max 50 chars)
- [ ] Store Name: Text input with autocomplete from previous stores
- [ ] Budget: Numeric input, formatted as currency
- [ ] Grocery Type: Dropdown (Weekly, Monthly, Special, Other)
- [ ] All fields are required except Grocery Type (defaults to Weekly)
- [ ] Only one active session allowed at a time
- [ ] Active session indicator shown on Home screen
- [ ] Error if attempting to start session while one is active

Data Stored (Authenticated):
```sql
grocery_sessions:
- id (UUID)
- user_id (UUID)
- session_name (TEXT)
- store_name (TEXT)
- spending_limit (DECIMAL)
- grocery_type (TEXT)
```

- is_active (BOOLEAN)
- created_at (TIMESTAMP)
- ended_at (TIMESTAMP)
```

**Data Stored (Guest):**
```javascript
AsyncStorage: guest_session
{
  sessionId: string,
  sessionName: string,
  storeName: string,
  spendingLimit: number,
  groceryType: string,
  isActive: boolean,
  createdAt: string
}
```

FR-2.2: View Active Session
Priority: P0 (Must Have)
User Story: *As a shopper, I want to see my active session details so that I remember my budget and store.*

Acceptance Criteria:
- [ ] Active session card displayed on Home screen
- [ ] Shows: Session name, Store, Budget, Time started
- [ ] Color-coded status indicator (green = active)
- [ ] Quick actions: View Cart, End Session
- [ ] Tap to view full session details

FR-2.3: End Session
Priority: P1 (Should Have)
User Story: *As a shopper, I want to end my session without saving if I didn't complete my shopping.*

Acceptance Criteria:
- [ ] "End Session" button in Cart tab and Session detail view
- [ ] Confirmation dialog: "Are you sure? Cart items will be cleared."
- [ ] For authenticated users: Sets `is_active: false`, adds `ended_at` timestamp
```

- [ ] For authenticated users: Deletes all items from `cart_items` table
- [ ] For guest users: Clears cart from AsyncStorage, marks session inactive
- [ ] No checkout record created
- [ ] User returned to Home screen
- [ ] Success message: "Session ended"

5.3 Product Scanning & Management

FR-3.1: Barcode Scanning
Priority: P0 (Must Have)
User Story: *As a shopper, I want to scan product barcodes so that I can quickly add items to my cart.*

Acceptance Criteria:
- [ ] Scan tab opens camera view automatically
- [ ] Camera viewfinder shows scan area overlay
- [ ] Supports UPC-A, UPC-E, EAN-13, EAN-8 barcode formats
- [ ] Audible beep on successful scan
- [ ] Scanned barcode displayed in form field
- [ ] User can manually enter barcode if camera unavailable
- [ ] Vibration feedback on scan (if device supports)
- [ ] Flash toggle button for low-light conditions
- [ ] Scan success rate > 95% in good lighting

Technical Notes:
- Use Expo Camera with barcode scanning
- Implement barcode validation
- Handle camera permissions gracefully

FR-3.2: Product Photo Capture
Priority: P0 (Must Have)
User Story: *As a shopper, I want to take photos of products so that I can remember what I bought.*

Acceptance Criteria:
- [ ] Camera switches to photo mode after barcode scan
- [ ] User can retake photo if unsatisfied
- [ ] Photo preview shown before confirming
- [ ] Image compressed to < 500KB for storage
- [ ] Photos uploaded to Supabase Storage (authenticated users)

- [ ] Photos stored locally (guest users)
- [ ] Fallback placeholder image if user skips photo

Technical Notes:
- Use Expo ImageManipulator for compression
- Resize images to max 800x800px
- Store images in Supabase Storage bucket: `product-images`

FR-3.3: Product Details Entry
Priority: P0 (Must Have)
User Story: *As a shopper, I want to enter product details so that my cart has accurate information.*

Acceptance Criteria:
- [ ] Form includes: Brand, Product Name, Price, Quantity
- [ ] Brand: Text input (max 50 chars)
- [ ] Product Name: Text input (max 100 chars)
- [ ] Price: Numeric input with currency formatting ($X.XX)
- [ ] Quantity: Stepper control (min: 1, max: 99)
- [ ] All fields are required
- [ ] Validation errors shown inline
- [ ] "Add to Cart" button disabled until form is valid
- [ ] Form clears after successful submission
- [ ] Success message shown briefly

FR-3.4: Add to Cart
Priority: P0 (Must Have)
User Story: *As a shopper, I want to add scanned products to my cart so that I can track my total.*

Acceptance Criteria:
- [ ] Tapping "Add to Cart" saves item to cart
- [ ] Cart badge shows item count in tab bar
- [ ] For authenticated users: Product saved to `products` table (if new)
- [ ] For authenticated users: Scan record saved to `scans` table
- [ ] For authenticated users: Cart item saved to `cart_items` table
- [ ] For guest users: Item saved to AsyncStorage cart
- [ ] Duplicate products with same barcode increment quantity
- [ ] Success animation shown after adding
- [ ] User can immediately scan another product

Data Stored (Authenticated):
```sql
products:
- id (UUID)
- barcode (TEXT, unique)
- name (TEXT)
- brand (TEXT)
- image_url (TEXT)
- created_at (TIMESTAMP)

scans:
- id (UUID)
- product_id (UUID)
- store_id (UUID)
- price (DECIMAL)
- scanned_at (TIMESTAMP)
- scanned_by (UUID - user_id)

cart_items:
- id (UUID)
- session_id (UUID)
- product_id (UUID)
- quantity (INTEGER)
- price (DECIMAL)
- added_at (TIMESTAMP)
```

Data Stored (Guest):
```javascript
AsyncStorage: guest_cart_{sessionId}
[
  {
    id: string,
    barcode: string,
    brand: string,
    name: string,
    price: number,
    quantity: number,
    imageUri: string,
    addedAt: string
  }
]
```

FR-3.5: Manual Product Entry
Priority: P1 (Should Have)
User Story: *As a shopper, I want to manually enter products without scanning so that I can add items without barcodes.*

Acceptance Criteria:
- [ ] "Enter Manually" button in Scan tab
- [ ] Shows same form as scan flow (Brand, Name, Price, Quantity)
- [ ] Barcode field optional
- [ ] Photo optional
- [ ] All other validation same as scan flow

5.4 Cart Management

FR-4.1: View Cart
Priority: P0 (Must Have)
User Story: *As a shopper, I want to view my cart so that I can see what I've added and my running total.*

Acceptance Criteria:
- [ ] Cart tab shows all items in active session
- [ ] Each item displays: Image, Brand, Name, Price, Quantity, Subtotal
- [ ] Session info at top: Name, Store, Budget
- [ ] Current total displayed prominently
- [ ] Budget progress bar (green < 90%, yellow 90-100%, red > 100%)
- [ ] Remaining budget shown (or overage if exceeded)
- [ ] Item count shown
- [ ] Empty state if no items: "Your cart is empty. Start scanning!"
- [ ] Real-time updates as items are added/removed

FR-4.2: Update Quantity
Priority: P0 (Must Have)
User Story: *As a shopper, I want to update item quantities so that my cart reflects the correct amounts.*

Acceptance Criteria:
- [ ] Plus (+) and minus (-) buttons on each cart item

- [ ] Quantity updates immediately in UI
- [ ] Total recalculates automatically
- [ ] Minimum quantity: 1
- [ ] Maximum quantity: 99
- [ ] Quantity of 0 prompts delete confirmation
- [ ] Changes saved to database (authenticated) or AsyncStorage (guest)
- [ ] Optimistic UI updates (no loading spinner for quantity changes)

FR-4.3: Delete Cart Item
Priority: P0 (Must Have)
User Story: *As a shopper, I want to remove items from my cart so that I can correct mistakes.*

Acceptance Criteria:
- [ ] Swipe left on item reveals delete button
- [ ] Tap delete button shows confirmation: "Remove [Product Name]?"
- [ ] Confirming removes item from cart
- [ ] Total recalculates automatically
- [ ] Undo option shown for 5 seconds after deletion
- [ ] Item removed from database/AsyncStorage
- [ ] Animation: Item slides out smoothly

FR-4.4: Edit Spending Limit
Priority: P1 (Should Have)
User Story: *As a shopper, I want to adjust my budget during shopping so that I can be flexible if needed.*

Acceptance Criteria:
- [ ] Tap on budget amount in Cart screen
- [ ] Shows input modal with current budget
- [ ] User enters new budget amount
- [ ] Budget updates in session record
- [ ] Progress bar recalculates based on new budget
- [ ] Change history logged (for future analytics)

FR-4.5: Clear Cart
Priority: P2 (Nice to Have)
User Story: *As a shopper, I want to clear my entire cart so that I can start over.*

Acceptance Criteria:
- [ ] "Clear Cart" button in Cart screen (in menu or toolbar)
- [ ] Confirmation dialog: "Are you sure? All items will be removed."
- [ ] Confirming removes all items from cart
- [ ] Session remains active (doesn't end session)
- [ ] Total resets to $0.00
- [ ] Success message: "Cart cleared"

5.5 Checkout

FR-5.1: Complete Checkout
Priority: P0 (Must Have)
User Story: *As a shopper, I want to complete my checkout so that my shopping trip is saved to history.*

Acceptance Criteria:
- [ ] "Complete Checkout" button in Cart screen (only enabled if cart has items)
- [ ] Confirmation dialog: "Complete your $XXX.XX shopping trip at [Store]?"
- [ ] Shows final total and item count
- [ ] Confirming triggers checkout process
- [ ] For authenticated users:
  - [ ] Creates record in `checkout_sessions` table
  - [ ] Copies all cart items to `checkout_items` table
  - [ ] Deletes all items from `cart_items` table
  - [ ] Deletes session from `grocery_sessions` table
- [ ] For guest users:
  - [ ] Saves checkout to AsyncStorage `guest_checkouts`
  - [ ] Clears cart from AsyncStorage
  - [ ] Marks session as complete
- [ ] Success screen shown with:
  - [ ] Confetti animation
  - [ ] "Shopping trip saved!"
  - [ ] Summary: Total spent, items, store
  - [ ] Buttons: "View History" or "Start New Session"
- [ ] Checkout completes in < 2 seconds

Data Stored (Authenticated):
```sql
checkout_sessions:
- id (UUID)
- user_id (UUID)
```

- store_id (UUID)
- session_name (TEXT)
- total_amount (DECIMAL)
- item_count (INTEGER)
- grocery_type (TEXT)
- checked_out_at (TIMESTAMP)

checkout_items:
- id (UUID)
- checkout_id (UUID)
- product_id (UUID)
- quantity (INTEGER)
- price (DECIMAL)
```

**Data Stored (Guest):**
```javascript
AsyncStorage: guest_checkouts
[
  {
    checkoutId: string,
    sessionName: string,
    storeName: string,
    totalAmount: number,
    itemCount: integer,
    groceryType: string,
    checkedOutAt: string,
    items: [...]
  }
]
```

5.6 History & Analytics

FR-6.1: View Shopping History
Priority: P1 (Should Have)
User Story: *As a shopper, I want to view my past shopping trips so that I can track my spending over time.*

Acceptance Criteria:
- [ ] History tab shows list of completed checkouts (authenticated users only)
- [ ] Each entry shows: Date, Store, Total, Item count
```

- [ ] Sorted by date (newest first)
- [ ] Tap to view detailed breakdown
- [ ] Detail view shows:
  - [ ] All items with images, names, quantities, prices
  - [ ] Store name and shopping date
  - [ ] Total amount
  - [ ] Grocery type (Weekly, Monthly, etc.)
- [ ] Search/filter options:
  - [ ] By store
  - [ ] By date range
  - [ ] By total range
- [ ] Empty state for guest users: "Sign up to save your shopping history"
- [ ] Export option (CSV or PDF)

FR-6.2: Analytics Dashboard
Priority: P2 (Nice to Have)
User Story: *As a data-driven shopper, I want to see analytics about my spending so that I can identify patterns and save money.*

Acceptance Criteria:
- [ ] Analytics tab shows charts and statistics (authenticated users only)
- [ ] Metrics displayed:
  - [ ] Total spent (this month, last month, all-time)
  - [ ] Average basket size
  - [ ] Most frequent store
  - [ ] Spending trend line chart (last 6 months)
  - [ ] Top 10 most purchased products
  - [ ] Price comparison by store (for same products)
- [ ] Date range selector (Last 30 days, 3 months, 6 months, Year, All)
- [ ] Visual charts using library (react-native-chart-kit or Victory Native)
- [ ] Empty state if insufficient data: "Shop more to see analytics!"
- [ ] Guest users see upsell message: "Sign up to unlock analytics"

FR-6.3: Price History
Priority: P2 (Nice to Have)
User Story: *As a deal hunter, I want to see price history for products so that I know when I'm getting a good deal.*

Acceptance Criteria:
- [ ] Tap on product in History detail view shows price history

- [ ] Line chart showing price over time
- [ ] Data points from all `scans` table records for that product
- [ ] Shows: Current price, lowest price, highest price, average price
- [ ] Color codes: Green if current < average, red if current > average
- [ ] Grouped by store (compare prices across stores)
- [ ] Minimum 3 data points required to show chart

---

6. Technical Architecture

6.1 Technology Stack

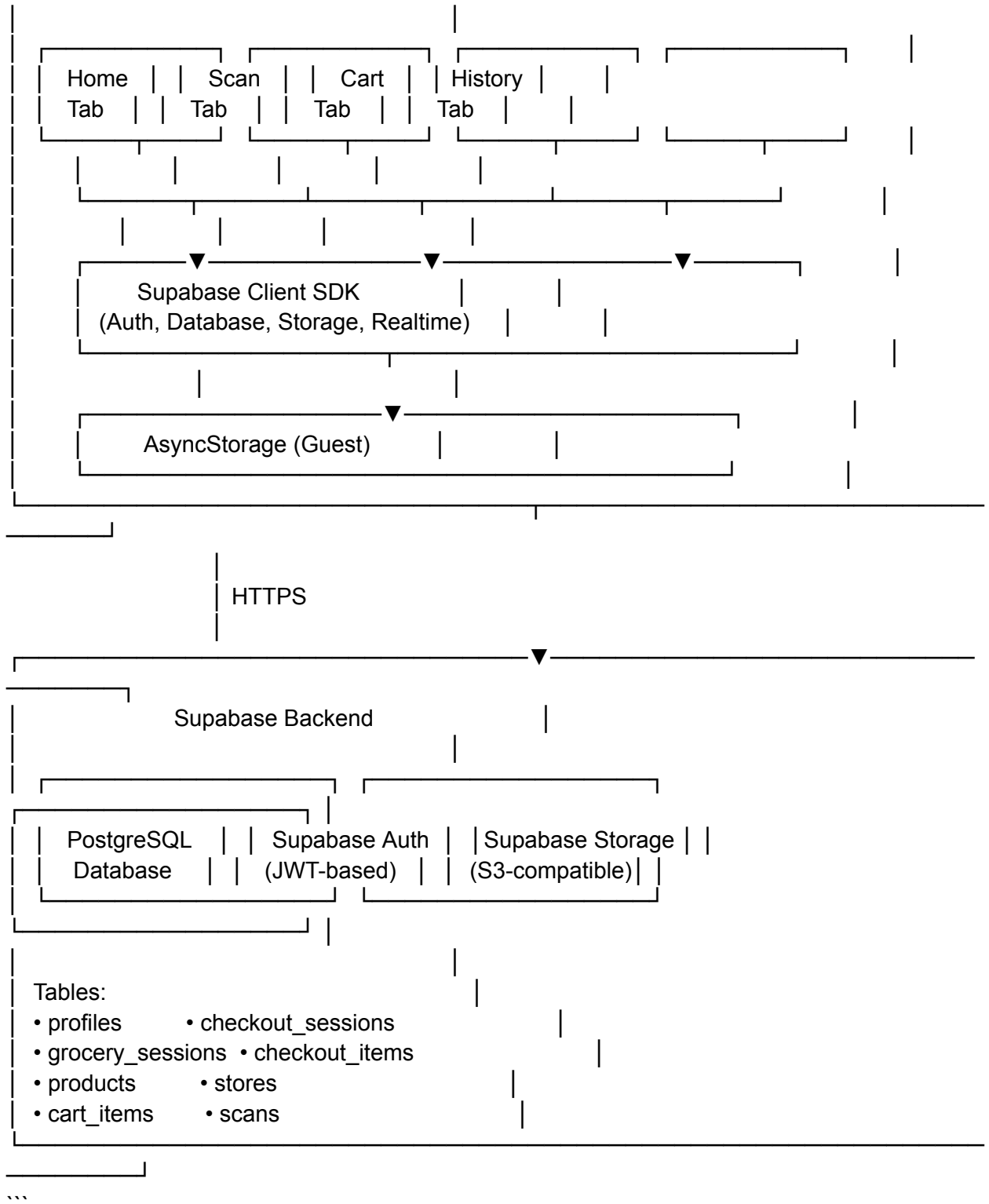
Frontend
- **Framework:** React Native (Expo)
- **Language:** TypeScript
- **Navigation:** Expo Router (file-based routing)
- **State Management:** React Context API + useState/useReducer
- **UI Components:** React Native Paper or NativeBase
- **Forms:** React Hook Form
- **Camera:** Expo Camera
- **Barcode Scanning:** expo-barcode-scanner
- **Local Storage:** AsyncStorage
- **Charts:** react-native-chart-kit or Victory Native

Backend
- **Database:** Supabase (PostgreSQL)
- **Authentication:** Supabase Auth
- **Storage:** Supabase Storage (for product images)
- **API:** Supabase Client SDK (REST + Realtime)

Deployment
- **iOS:** Apple App Store
- **Android:** Google Play Store
- **Build Service:** EAS Build (Expo)

6.2 System Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────────────────────────┐
│                                  Mobile App (React Native)                               │
```


Home Tab
Scan Tab
Cart Tab
History Tab
Supabase Client SDK
(Auth, Database, Storage, Realtime)
AsyncStorage (Guest)
HTTPS
Supabase Backend
PostgreSQL Database
Supabase Auth
(JWT-based)
Supabase Storage
(S3-compatible)
Tables:
• profiles • checkout_sessions
• grocery_sessions • checkout_items
• products • stores
• cart_items • scans

```

---

### 6.3 Data Flow
```

User Authentication Flow
```
User Taps Login
   ↓
Supabase Auth API
   ↓
JWT Token Generated
   ↓
Stored in Secure Storage
   ↓
All API Calls Include Token
   ↓
RLS Policies Enforce Access Control
```

Product Scanning Flow
```
User Scans Barcode
   ↓
expo-barcode-scanner Detects Code
   ↓
Check if Product Exists (products table)
   ↓
If New: Create Product Record
   ↓
User Takes Photo
   ↓
Upload to Supabase Storage
   ↓
User Enters Details
   ↓
Create Scan Record (scans table)
   ↓
Create Cart Item (cart_items table)
   ↓
Update UI with New Total
```

Checkout Flow
```
User Taps Complete Checkout
   ↓
Create checkout_sessions Record
```

```
   ↓
Copy cart_items → checkout_items
   ↓
Delete cart_items Records
   ↓
Delete grocery_sessions Record
   ↓
Show Success Screen
   ↓
Navigate to History
```

6.4 State Management Strategy

Global State (Context API)
- **AuthContext:** User authentication state, login/logout functions
- **SessionContext:** Active grocery session data
- **CartContext:** Cart items, total, quantity functions

Local State (useState)
- Form inputs (product details, session setup)
- UI state (modals, loading spinners, error messages)
- Camera state (flash on/off, camera ready)

Server State (Supabase Realtime)
- Cart items (realtime updates across devices)
- Active session (realtime sync if shopping with partner)

6.5 API Endpoints (Supabase)

All API calls use Supabase Client SDK with automatic authentication via JWT tokens.

Authentication
- `supabase.auth.signUp({ email, password })`
- `supabase.auth.signIn({ email, password })`
- `supabase.auth.signOut()`
- `supabase.auth.resetPasswordForEmail(email)`

Grocery Sessions
- `GET /grocery_sessions?user_id=eq.{userId}&is_active=eq.true`

- `POST /grocery_sessions` (Create new session)
- `PATCH /grocery_sessions?id=eq.{sessionId}` (Update session)
- `DELETE /grocery_sessions?id=eq.{sessionId}` (Delete session)

Cart Items
- `GET /cart_items?session_id=eq.{sessionId}`
- `POST /cart_items` (Add to cart)
- `PATCH /cart_items?id=eq.{itemId}` (Update quantity)
- `DELETE /cart_items?id=eq.{itemId}` (Remove from cart)

Products
- `GET /products?barcode=eq.{barcode}`
- `POST /products` (Create product)

Checkouts
- `GET /checkout_sessions?user_id=eq.{userId}`
- `POST /checkout_sessions` (Create checkout)
- `GET /checkout_items?checkout_id=eq.{checkoutId}`

Scans (Price History)
- `GET /scans?product_id=eq.{productId}`
- `POST /scans` (Record scan)

6.6 Security Considerations

Row-Level Security (RLS) Policies

profiles table:
```sql
-- Users can only read/update their own profile
CREATE POLICY "Users can view own profile"
ON profiles FOR SELECT
USING (auth.uid() = id);

CREATE POLICY "Users can update own profile"
ON profiles FOR UPDATE
USING (auth.uid() = id);
```

grocery_sessions table:
```sql
-- Users can only access their own sessions
```

```
CREATE POLICY "Users can manage own sessions"
ON grocery_sessions FOR ALL
USING (auth.uid() = user_id);
```

cart_items table:
```sql
-- Users can only access cart items in their own sessions
CREATE POLICY "Users can manage own cart items"
ON cart_items FOR ALL
USING (
  auth.uid() = (
    SELECT user_id FROM grocery_sessions
    WHERE id = cart_items.session_id
  )
);
```

checkout_sessions & checkout_items:
```sql
-- Users can only access their own checkouts
CREATE POLICY "Users can view own checkouts"
ON checkout_sessions FOR SELECT
USING (auth.uid() = user_id);
```

Data Validation
- All user inputs sanitized before database insertion
- Numeric fields validated (price, quantity must be > 0)
- Email validation on registration
- Barcode format validation

Image Storage Security
- Product images uploaded to authenticated-only bucket
- Image size limited to 5MB
- File type restricted to: JPEG, PNG, WebP
- Image URLs signed with expiration (1 hour)

---

7. Data Models

7.1 Database Schema (PostgreSQL)

profiles
User profile information (automatically created on signup)

```sql
CREATE TABLE profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  email TEXT UNIQUE NOT NULL,
  full_name TEXT,
  avatar_url TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

---

stores
Store information (dynamically created from user input)

```sql
CREATE TABLE stores (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name TEXT NOT NULL,
  address TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),

  UNIQUE(name)
);
```

---

products
Product catalog (dynamically created from scans)

```sql
CREATE TABLE products (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  barcode TEXT UNIQUE NOT NULL,
  name TEXT NOT NULL,
  brand TEXT,
  image_url TEXT,
  category TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
```

```
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_products_barcode ON products(barcode);
```

---

grocery_sessions
Active shopping sessions

```sql
CREATE TABLE grocery_sessions (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  session_name TEXT NOT NULL,
  store_name TEXT NOT NULL,
  spending_limit DECIMAL(10,2) NOT NULL,
  grocery_type TEXT NOT NULL DEFAULT 'Weekly',
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  ended_at TIMESTAMPTZ,

  CHECK (spending_limit > 0)
);

CREATE INDEX idx_grocery_sessions_user_id ON grocery_sessions(user_id);
CREATE INDEX idx_grocery_sessions_is_active ON grocery_sessions(is_active);
```

---

cart_items
Items in current shopping cart

```sql
CREATE TABLE cart_items (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  session_id UUID NOT NULL REFERENCES grocery_sessions(id) ON DELETE CASCADE,
  product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  quantity INTEGER NOT NULL DEFAULT 1,
  price DECIMAL(10,2) NOT NULL,
  added_at TIMESTAMPTZ DEFAULT NOW(),
```

```
  CHECK (quantity > 0),
  CHECK (price >= 0),
  UNIQUE(session_id, product_id)
);

CREATE INDEX idx_cart_items_session_id ON cart_items(session_id);
```

---

scans
Historical price data for products across stores

```sql
CREATE TABLE scans (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  store_id UUID REFERENCES stores(id) ON DELETE SET NULL,
  store_name TEXT NOT NULL,
  price DECIMAL(10,2) NOT NULL,
  scanned_by UUID REFERENCES profiles(id) ON DELETE SET NULL,
  scanned_at TIMESTAMPTZ DEFAULT NOW(),

  CHECK (price >= 0)
);

CREATE INDEX idx_scans_product_id ON scans(product_id);
CREATE INDEX idx_scans_store_id ON scans(store_id);
CREATE INDEX idx_scans_scanned_at ON scans(scanned_at);
```

---

checkout_sessions
Completed shopping trips

```sql
CREATE TABLE checkout_sessions (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  store_id UUID REFERENCES stores(id) ON DELETE SET NULL,
  store_name TEXT NOT NULL,
  session_name TEXT NOT NULL,
  total_amount DECIMAL(10,2) NOT NULL,
```

```
  item_count INTEGER NOT NULL,
  grocery_type TEXT NOT NULL,
  checked_out_at TIMESTAMPTZ DEFAULT NOW(),

  CHECK (total_amount >= 0),
  CHECK (item_count > 0)
);

CREATE INDEX idx_checkout_sessions_user_id ON checkout_sessions(user_id);
CREATE INDEX idx_checkout_sessions_checked_out_at ON
checkout_sessions(checked_out_at);
```

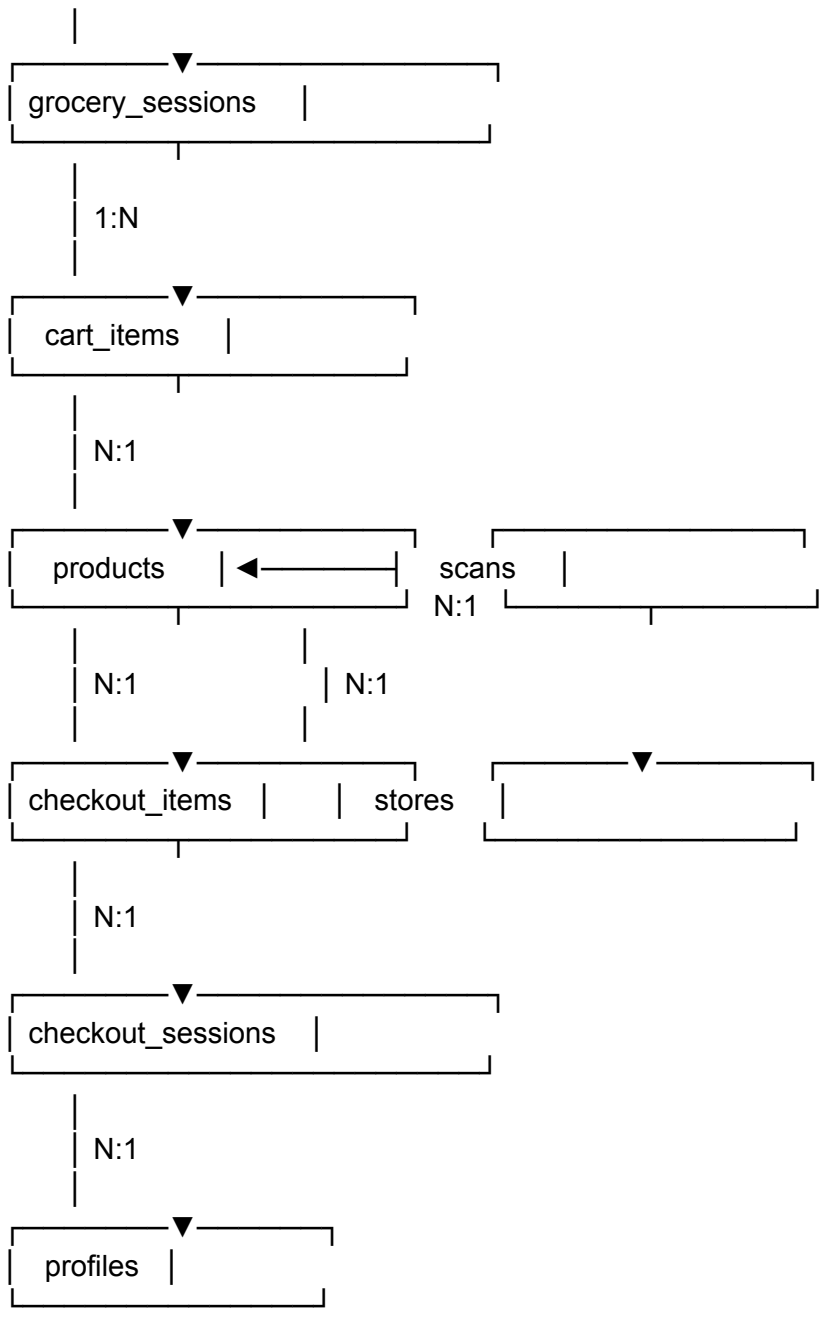
---

checkout_items
Items from completed shopping trips

```sql
CREATE TABLE checkout_items (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  checkout_id UUID NOT NULL REFERENCES checkout_sessions(id) ON DELETE CASCADE,
  product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  quantity INTEGER NOT NULL,
  price DECIMAL(10,2) NOT NULL,

  CHECK (quantity > 0),
  CHECK (price >= 0)
);

CREATE INDEX idx_checkout_items_checkout_id ON checkout_items(checkout_id);
CREATE INDEX idx_checkout_items_product_id ON checkout_items(product_id);
```

7.2 Database Relationships Diagram

```
┌──────────────────┐
│   profiles   │
└──────┬───────┘
       │
       │ 1:N
```



7.3 AsyncStorage Schema (Guest Users)

guest_session
```typescript
interface GuestSession {
  sessionId: string;
```

```
  sessionName: string;
  storeName: string;
  spendingLimit: number;
  groceryType: string;
  isActive: boolean;
  createdAt: string; // ISO 8601
}
```

guest_cart_{sessionId}
```typescript
interface GuestCartItem {
  id: string;
  barcode: string;
  brand: string;
  name: string;
  price: number;
  quantity: number;
  imageUri: string; // Local file URI
  addedAt: string; // ISO 8601
}

type GuestCart = GuestCartItem[];
```

guest_checkouts
```typescript
interface GuestCheckout {
  checkoutId: string;
  sessionName: string;
  storeName: string;
  totalAmount: number;
  itemCount: number;
  groceryType: string;
  checkedOutAt: string; // ISO 8601
  items: GuestCartItem[];
}

type GuestCheckouts = GuestCheckout[];
```

---

8. User Interface Specifications

8.1 Design Principles
- **Simplicity First:** Minimize cognitive load, clear hierarchy
- **Thumb-Friendly:** Bottom navigation, large tap targets (min 44x44pt)
- **Visual Feedback:** Animations for actions (add to cart, delete, etc.)
- **Accessibility:** High contrast, readable fonts (min 16pt), VoiceOver support
- **Consistent:** Reusable components, consistent spacing (8pt grid)

8.2 Color Palette

Primary Colors
- Primary: `#007AFF` (iOS Blue)
- Secondary: `#34C759` (Success Green)
- Accent: `#FF9500` (Warning Orange)

Semantic Colors
- Success: `#34C759`
- Warning: `#FF9500`
- Error: `#FF3B30`
- Info: `#007AFF`

Neutral Colors
- Background: `#FFFFFF` / `#000000` (Light/Dark mode)
- Surface: `#F2F2F7` / `#1C1C1E`
- Text Primary: `#000000` / `#FFFFFF`
- Text Secondary: `#8E8E93`
- Border: `#C6C6C8`

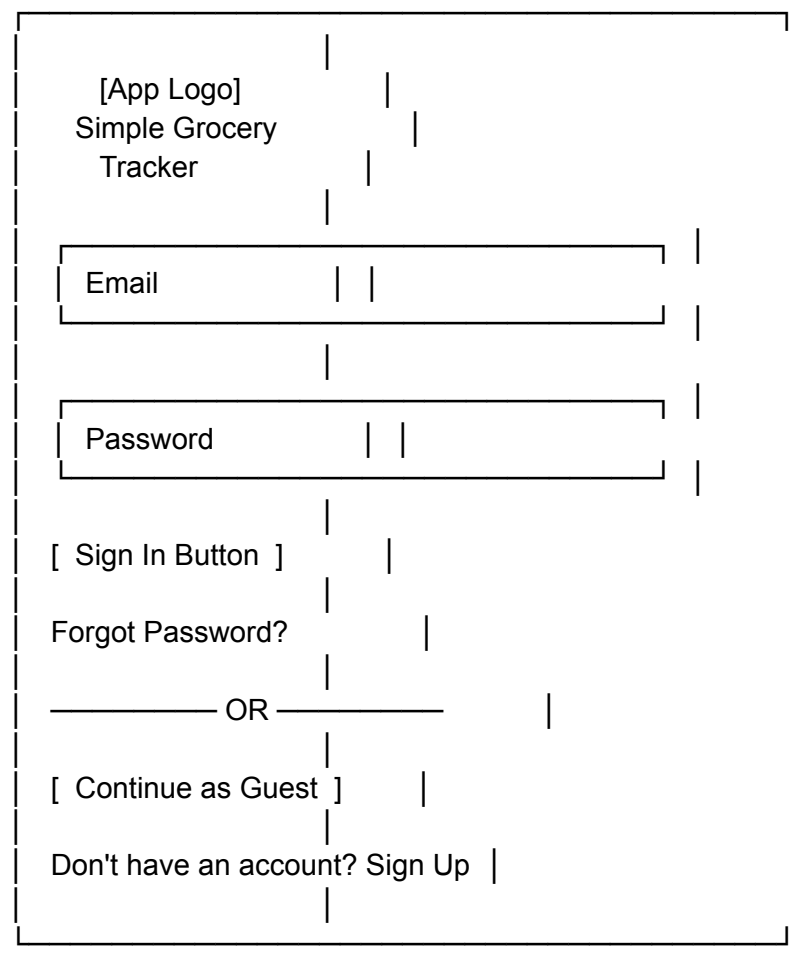
8.3 Typography

Font Family
- **iOS:** SF Pro (System Font)
- **Android:** Roboto (System Font)

Type Scale
- **H1:** 34pt, Bold (Screen titles)
- **H2:** 28pt, Bold (Section headers)
- **H3:** 22pt, Semibold (Card titles)
- **Body:** 17pt, Regular (Body text)
- **Caption:** 13pt, Regular (Secondary text)
- **Button:** 17pt, Semibold (Buttons)

8.4 Screen Layouts

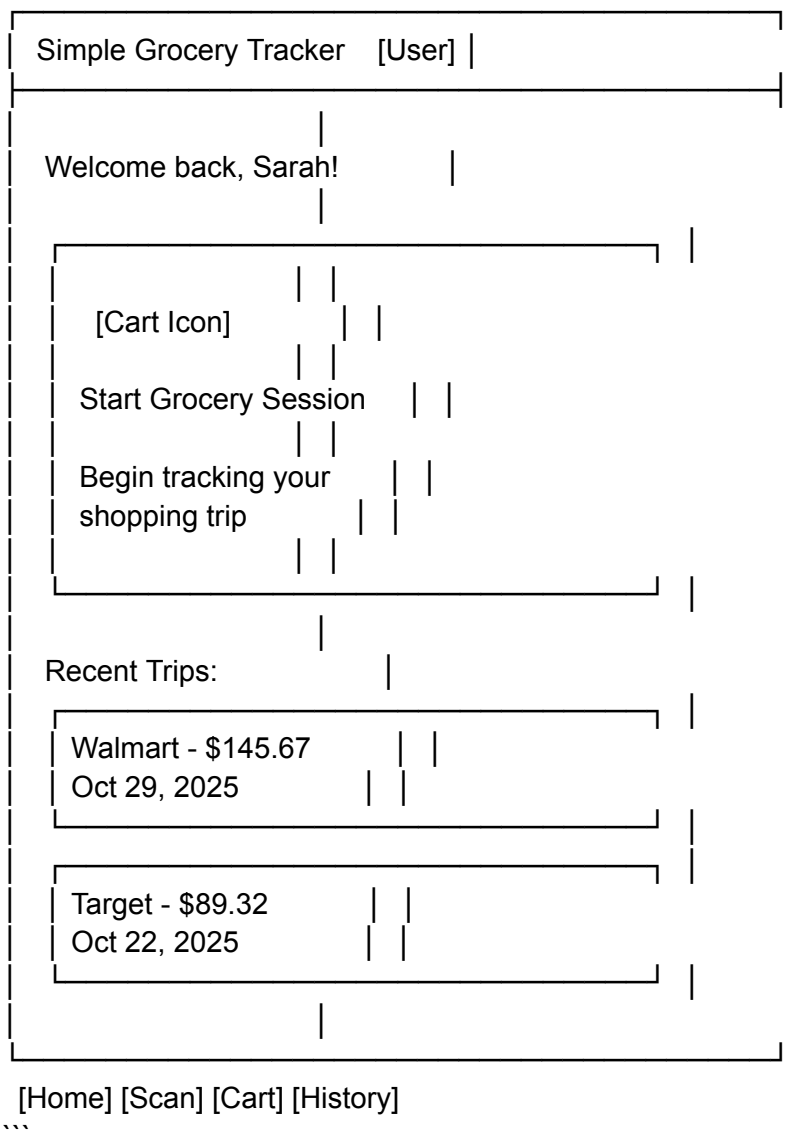
8.4.1 Login Screen

Layout:


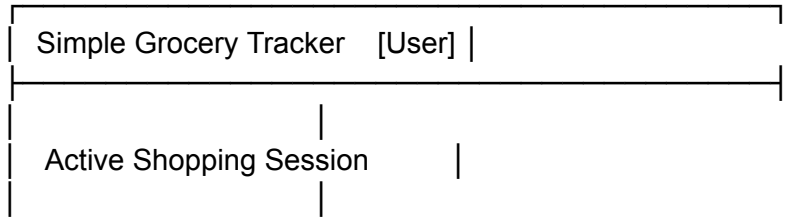
Components:
- App logo (150x150pt)
- Email input (rounded corners, icon)
- Password input (show/hide toggle)
- Primary button: "Sign In"
- Secondary button: "Continue as Guest"
- Text links: Forgot Password, Sign Up

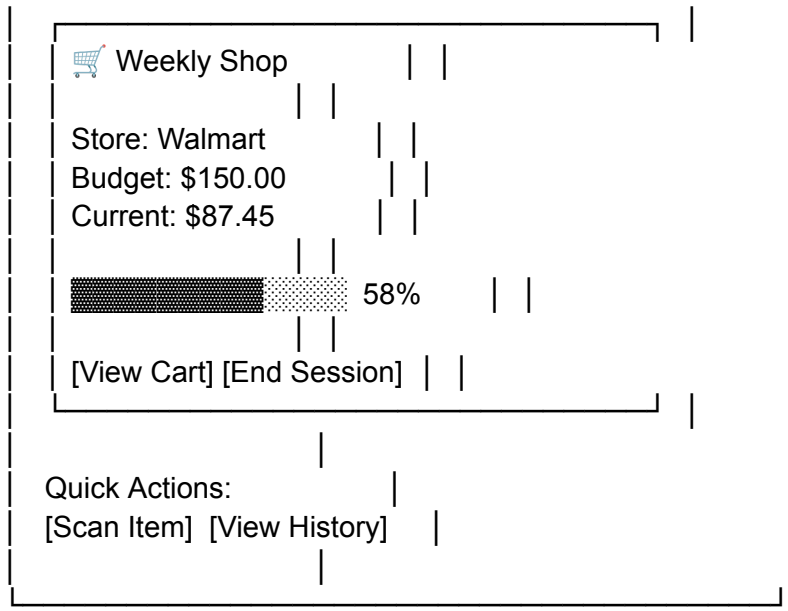
8.4.2 Home Screen

Layout (No Active Session):



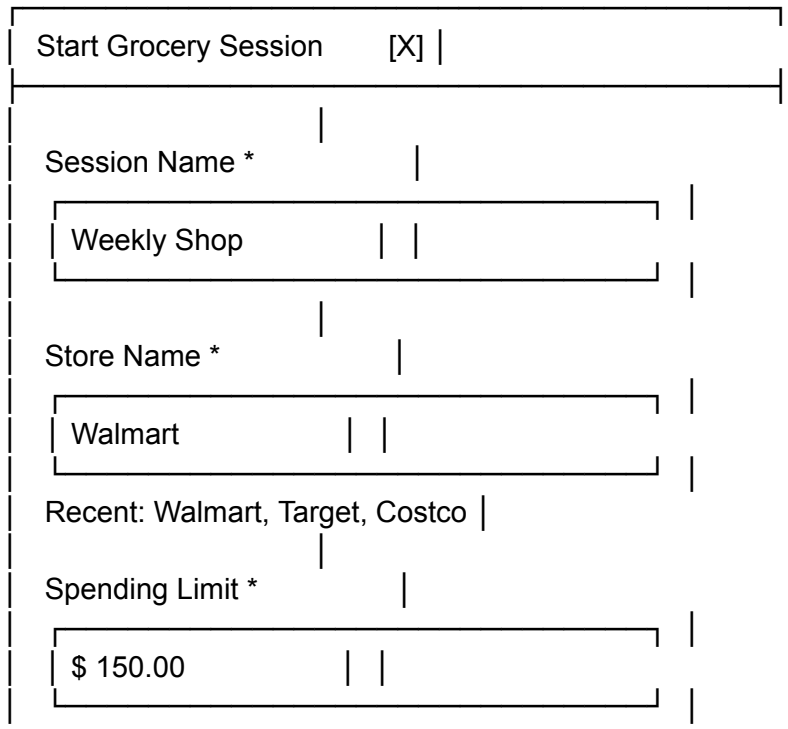
Layout (Active Session):

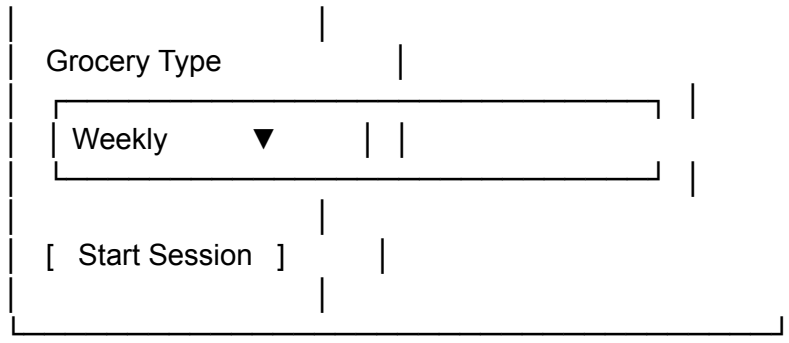




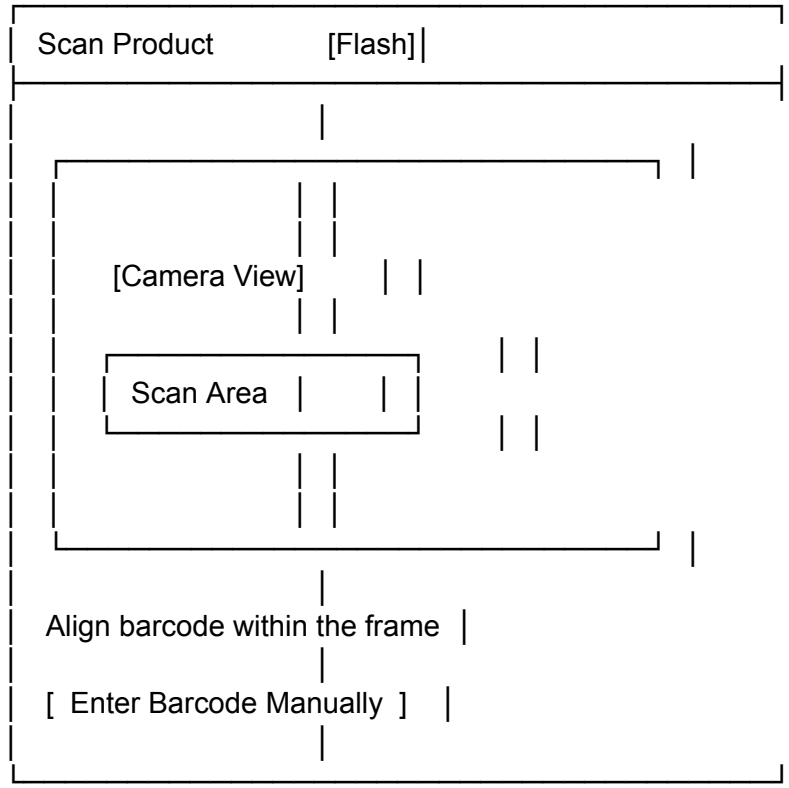
8.4.3 Start Session Modal

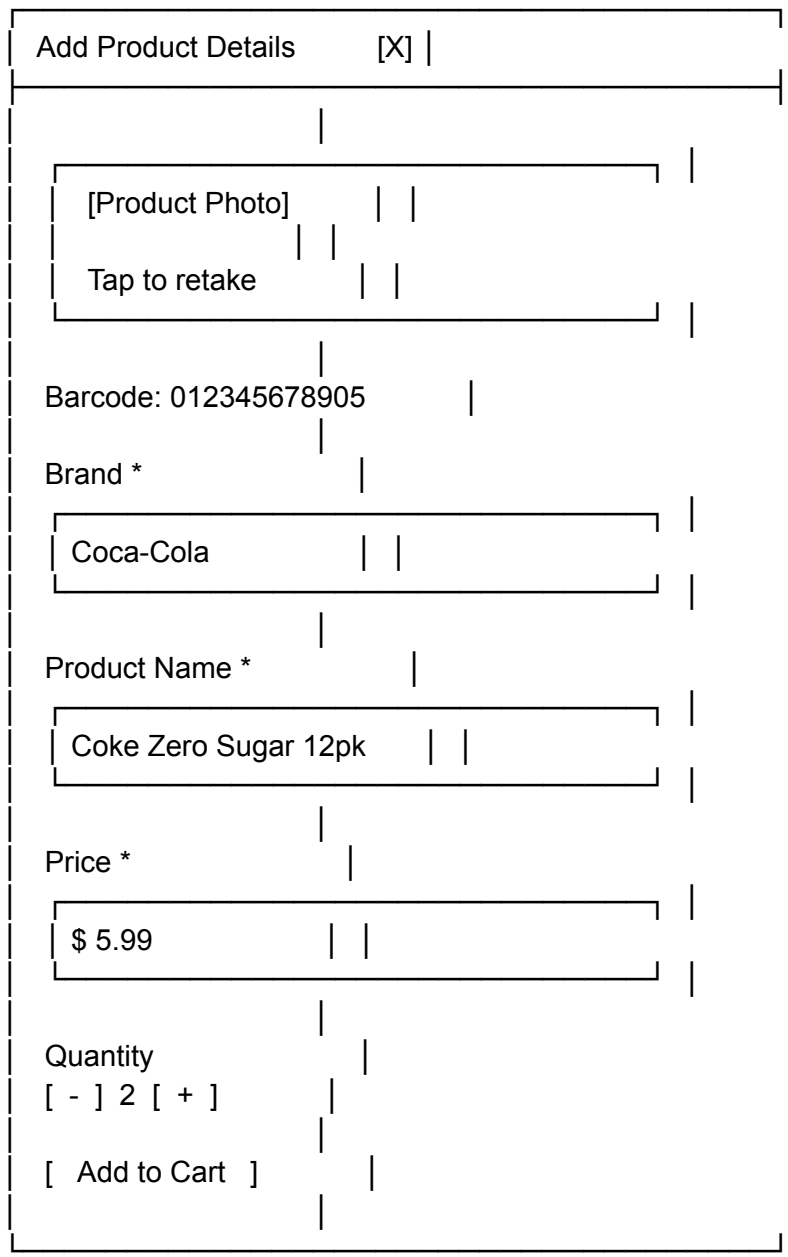
Layout:






```

**Validation:**
- All fields required except Grocery Type
- Spending limit must be > $0
- Session name max 50 characters

---

#### **8.4.4 Scan Screen**

**Layout (Camera Active):**
```



```

**Layout (Product Details Form):**
```


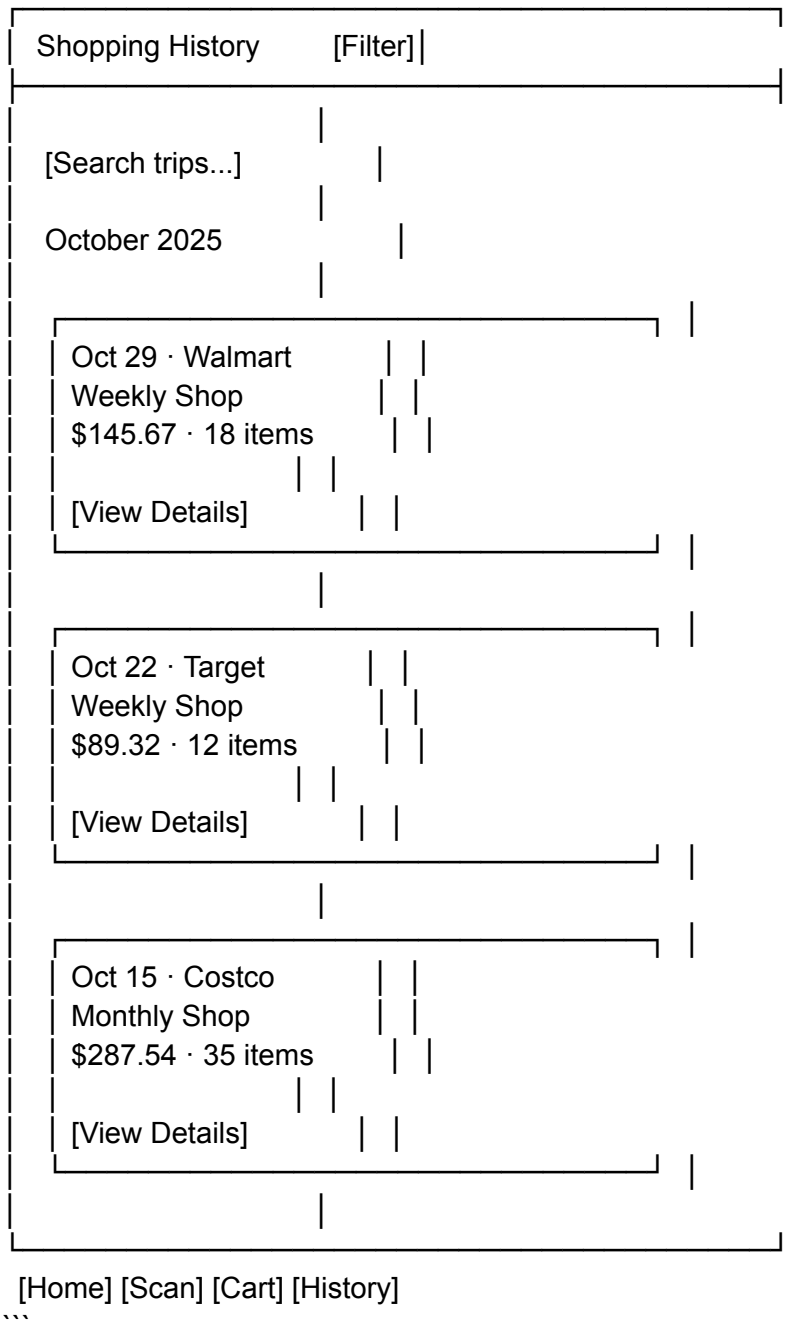
```

---

#### **8.4.5 Cart Screen**

**Layout:**
```

```
┌─────────────────────────────────┐
│ Cart                  [Menu] │
├─────────────────────────────────┤
│                              │
│  Weekly Shop - Walmart       │
│  Budget: $150.00             │
│                              │
│  ████████████░░ 87%          │
│                              │
│  Current: $130.45            │
│  Remaining: $19.55           │
│                              │
├─────────────────────────────────┤
│                              │
│  ┌─────┬─────────────────┬────┐ │
│  │[Img]│ Coca-Cola     │$12 │ │
│  │     │ Coke Zero 12pk │    │ │
│  │     │ $5.99 × 2      │ -2+│ │
│  └─────┴─────────────────┴────┘ │
│                              │
│  ┌─────┬─────────────────┬────┐ │
│  │[Img]│ Wonder Bread   │$3  │ │
│  │     │ White Bread    │    │ │
│  │     │ $2.99 × 1      │ -1+│ │
│  └─────┴─────────────────┴────┘ │
│                              │
│  [... 8 more items ...]      │
│                              │
├─────────────────────────────────┤
│                              │
│  Total: $130.45              │
│  Items: 12                   │
│                              │
│  [ Complete Checkout ]       │
│  [  End Session  ]           │
│                              │
└─────────────────────────────────┘
  [Home] [Scan] [Cart] [History]
```

Interactive Elements:
- Swipe left on item → Delete button appears
- Tap +/- buttons → Quantity updates, total recalculates

- Tap budget bar → Edit spending limit modal
- Tap Complete Checkout → Confirmation dialog

8.4.6 History Screen

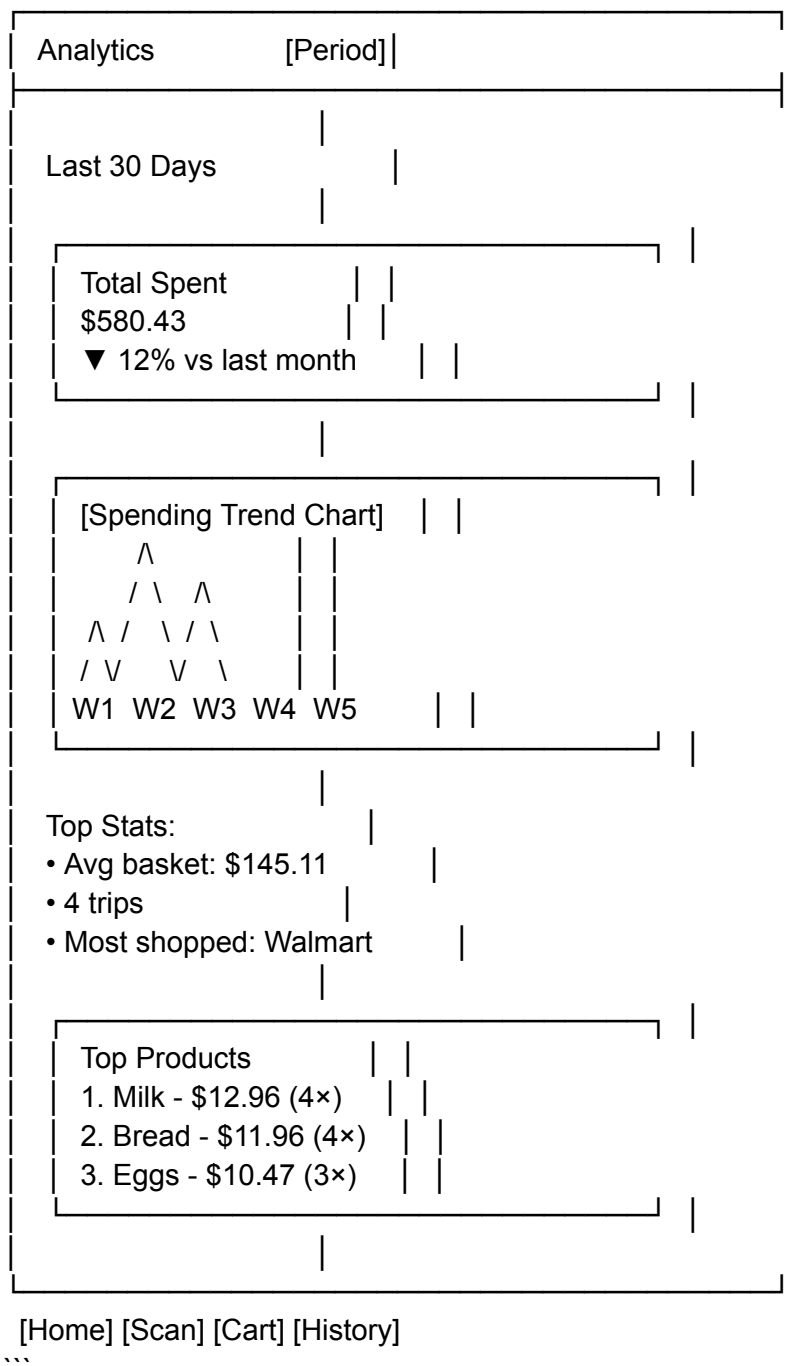
Layout:


8.4.7 Checkout Detail Screen

Layout:
```
┌──────────────────────────────────────────────────┐
│ ← Weekly Shop                    │
├──────────────────────────────────────────────────┤
│                          │
│  Walmart                       │
│  October 29, 2025 · 3:42 PM      │
│                          │
│  Total: $145.67                  │
│  Items: 18                     │
│                          │
├──────────────────────────────────────────────────┤
│                          │
│  Items Purchased:                │
│                          │
│  ┌──────┬──────────────────────────┬─────┐ │
│  │[Img]│ Coca-Cola        │$12 │ │
│  │     │ Coke Zero 12pk   │    │ │
│  │     │ $5.99 × 2        │    │ │
│  └──────┴──────────────────────────┴─────┘ │
│                          │
│  ┌──────┬──────────────────────────┬─────┐ │
│  │[Img]│ Wonder Bread     │$3  │ │
│  │     │ White Bread      │    │ │
│  │     │ $2.99 × 1        │    │ │
│  └──────┴──────────────────────────┴─────┘ │
│                          │
│  [... 16 more items ...]         │
│                          │
│  [ Export Receipt ]              │
│                          │
└──────────────────────────────────────────────────┘
```

8.4.8 Analytics Screen

Layout:



8.5 Component Library

Buttons

- **Primary:** Filled, bold, primary color
- **Secondary:** Outlined, primary color border
- **Tertiary:** Text only, primary color text
- **Destructive:** Filled, error color

Input Fields
- **Text Input:** Rounded corners (8pt), border, icon support
- **Numeric Input:** Keyboard: decimal-pad, currency formatting
- **Dropdown:** Native picker with chevron icon
- **Stepper:** -/+ buttons with centered number

Cards
- **Elevation:** Shadow (iOS) / Elevation (Android)
- **Corners:** 12pt radius
- **Padding:** 16pt
- **Background:** Surface color

Progress Bars
- **Linear:** Rounded ends, color changes based on percentage
  - Green: 0-89%
  - Yellow: 90-99%
  - Red: 100%+

Badges
- **Cart Badge:** Circular, primary color, white text
- **Status Badge:** Small, color-coded (green=active, gray=ended)

---

9. Non-Functional Requirements

9.1 Performance

Response Time:
- App launch: < 2 seconds
- Screen transitions: < 300ms
- Database queries: < 500ms
- Image uploads: < 3 seconds
- Barcode scan recognition: < 1 second

Scalability:
- Support 10,000+ concurrent users
- Handle 1M+ products in database
- Handle 100+ items in single cart

- Handle 1000+ checkout history records per user

Optimization:
- Lazy loading for history lists
- Image caching for product photos
- Pagination for large datasets
- Database indexes on foreign keys and commonly queried fields

---

9.2 Reliability

Uptime:
- Target: 99.5% uptime (43 hours downtime/year)
- Supabase SLA: 99.9% for paid plans

Data Integrity:
- Automatic database backups (daily)
- Transaction rollbacks on checkout errors
- Validation on all user inputs
- Row-level security enforced

Error Handling:
- Graceful degradation if camera unavailable
- Offline mode for guest users (AsyncStorage)
- Retry logic for failed API calls
- User-friendly error messages (no technical jargon)

9.3 Usability

Accessibility:
- WCAG 2.1 Level AA compliance
- VoiceOver/TalkBack support
- Minimum tap target: 44x44pt
- Minimum text size: 16pt
- High contrast mode support
- Dynamic type support (iOS)

Learnability:
- Onboarding tutorial on first launch (optional)
- Contextual tooltips for key features
- Empty states with instructions

- Inline validation with helpful messages

Efficiency:
- Average time to add product: < 10 seconds
- Average time to complete checkout: < 5 seconds
- Keyboard remains open during multi-field forms
- Smart defaults (e.g., quantity = 1)

9.4 Security

Data Protection:
- Passwords hashed with bcrypt
- HTTPS for all API calls
- JWT tokens for authentication
- Row-level security on all tables
- Image URLs signed with expiration

Privacy:
- Guest mode: No data sent to server
- GDPR compliant: User can delete all data
- No third-party analytics without consent
- Privacy policy clearly displayed

Compliance:
- PCI DSS: Not required (no payment processing)
- GDPR: Right to access, delete, export data
- CCPA: California privacy compliance

9.5 Maintainability

Code Quality:
- TypeScript for type safety
- ESLint + Prettier for code formatting
- Component-based architecture (reusable)
- Unit tests for critical functions (80% coverage)
- Integration tests for key flows (checkout, scan)

Documentation:
- Inline code comments
- README with setup instructions

- API documentation (Swagger/OpenAPI)
- Architecture decision records (ADRs)

Monitoring:
- Error tracking (Sentry)
- Performance monitoring (Firebase Performance)
- Analytics (Mixpanel or Amplitude)
- User feedback mechanism (in-app)

9.6 Compatibility

Platforms:
- iOS: 13.0+
- Android: 8.0 (API 26)+

Devices:
- iPhone 8 and newer
- iPad (responsive layout)
- Android phones with camera
- Android tablets (responsive layout)

Network:
- Works on WiFi and cellular (3G/4G/5G)
- Graceful degradation on slow connections
- Offline mode for guest users

---

10. Success Metrics

10.1 Product Metrics (KPIs)

Acquisition:
- 10,000 app downloads in first 3 months
- 30% conversion rate from guest → registered user
- 4.0+ star rating on app stores
- < 5% uninstall rate within first 30 days

Engagement:
- 70% of users complete at least 1 checkout
- Average 2.5 sessions per week
- Average session duration: 15+ minutes

- 60% retention rate at 30 days

Revenue (Future):
- Not applicable for V1 (free app)
- Future: Premium features, ads, affiliate links

10.2 Usage Metrics

Session Activity:
- Average products scanned per session: 15
- Average checkout value: $120
- Average session duration: 18 minutes
- Sessions ending in checkout: 85%

Feature Adoption:
- % of users using barcode scan: 90%
- % of users using manual entry: 30%
- % of users viewing history: 60%
- % of users viewing analytics: 40%

10.3 Technical Metrics

Performance:
- Average API response time: < 500ms
- Image upload success rate: > 95%
- Barcode scan success rate: > 90%
- App crash rate: < 1%

Quality:
- Test coverage: > 80%
- Production bugs per release: < 5 critical
- Average bug resolution time: < 48 hours

10.4 Measurement Tools

Analytics:
- Mixpanel or Amplitude (user behavior)
- Firebase Analytics (app usage)

- Google Analytics 4 (web-based reporting)

Error Tracking:
- Sentry (crash reporting, error logs)

User Feedback:
- In-app feedback form
- App Store reviews monitoring
- NPS surveys (quarterly)

---

11. Release Plan

11.1 MVP (Version 1.0) - Target: 8 Weeks

Phase 1: Core Backend (Week 1-2)
- ✅ Supabase project setup
- ✅ Database schema creation
- ✅ Row-level security policies
- ✅ Authentication flow
- ✅ Image storage bucket

Phase 2: Core Frontend (Week 3-5)
- ✅ App navigation structure (Expo Router)
- ✅ Authentication screens (Login, Sign Up, Guest)
- ✅ Home screen + Session management
- ✅ Scan screen + Camera integration
- ✅ Product form + Image capture
- ✅ Cart screen + Quantity management
- ✅ Checkout flow

Phase 3: History & Polish (Week 6-7)
- ✅ History screen
- ✅ Checkout detail view
- ✅ UI/UX refinements
- ✅ Error handling
- ✅ Loading states
- ✅ Empty states

Phase 4: Testing & Launch (Week 8)
- ✅ Beta testing (TestFlight, Google Play Beta)
- ✅ Bug fixes
- ✅ App Store assets (screenshots, description)

- ✅ App submission
- ✅ Public launch

MVP Feature Checklist:
- [x] User authentication (email/password)
- [x] Guest mode
- [x] Start/end grocery session
- [x] Barcode scanning
- [x] Product photo capture
- [x] Manual product entry
- [x] Add to cart
- [x] View cart
- [x] Update quantities
- [x] Delete items
- [x] Complete checkout
- [x] View history
- [x] Checkout detail view

11.2 Version 1.1 - Target: +4 Weeks

Features:
- Analytics dashboard (spending trends, top products)
- Store price comparison (same product across stores)
- Search & filter in history
- Export receipts (PDF)
- Social login (Google, Apple)
- Dark mode support

11.3 Version 1.2 - Target: +8 Weeks

Features:
- Shopping lists (pre-trip planning)
- Budget alerts (push notifications)
- Price drop alerts (products on sale)
- Multi-user sessions (shopping with partner)
- Barcode database integration (automatic product info)
- Receipt scanning (OCR for bulk import)

11.4 Version 2.0 - Target: +6 Months

Major Features:
- Premium subscription tier
- Advanced analytics (ML-based insights)
- Meal planning integration
- Store navigation (in-store maps)
- Loyalty card integration
- Cashback/rewards program

---

12. Future Enhancements

12.1 Short-Term (3-6 Months)

Improved Scanning:
- AI product recognition (identify products without barcode)
- Bulk scanning mode (scan multiple items quickly)
- Voice input for product details

Better History:
- Compare two shopping trips side-by-side
- Budget vs. actual charts
- Monthly spending reports (PDF export)

Social Features:
- Share shopping lists with family
- Split costs with roommates
- Shared family budget tracking

12.2 Medium-Term (6-12 Months)

Smart Features:
- AI budget recommendations based on history
- Predictive shopping lists (based on purchase patterns)
- Price prediction (best time to buy)
- Store recommendations (cheapest for your list)

Integrations:
- Grocery store loyalty cards (Walmart+, Target Circle)
- Recipe apps (import ingredients as shopping list)

- Meal kit services (import as cart items)

Platform Expansion:
- Web app (responsive PWA)
- Smartwatch app (Apple Watch, Wear OS)
- Browser extension (price tracking)

12.3 Long-Term (12+ Months)

Advanced Analytics:
- Nutritional analysis (based on purchased products)
- Carbon footprint tracking (sustainability)
- Waste reduction insights (unused purchases)

Monetization:
- Premium tier ($4.99/month)
  - Unlimited history storage
  - Advanced analytics
  - Price drop alerts
  - Ad-free experience
- Affiliate revenue (product recommendations)
- In-app ads (free tier)

Enterprise:
- B2B version for personal shoppers
- API for third-party integrations
- White-label solution for grocery chains

---

13. Risks & Mitigation

13.1 Technical Risks

Risk	Probability	Impact	Mitigation
Camera API inconsistencies across devices	Medium	High	Extensive device testing, fallback to manual entry
Barcode scan accuracy issues	Medium	High	Multiple barcode libraries, manual override option
Image storage costs exceed budget	Low	Medium	Image compression, usage limits for free tier

| Supabase performance issues at scale | Low | High | Caching strategy, database optimization, migration plan |
| App Store rejection | Medium | High | Follow guidelines closely, beta testing, pre-submission review |

13.2 Product Risks

Risk	Probability	Impact	Mitigation
Low user adoption	Medium	High	Marketing strategy, influencer partnerships, referral program
Users abandon guest mode (don't convert)	High	Medium	Prompt at key moments, show value of account
Feature complexity overwhelms users	Medium	High	Progressive disclosure, optional tutorial, simple MVP
Competitors copy features	High	Low	Focus on UX, build community, iterate quickly

---

13.3 Business Risks

Risk	Probability	Impact	Mitigation
Insufficient funding for development	Low	High	Phased release, MVP first, validate before expanding
Legal issues (privacy, data protection)	Low	High	Legal review, privacy policy, GDPR compliance
Dependency on Supabase (vendor lock-in)	Medium	Medium	Abstract data layer, migration plan, regular backups

---

14. Appendix

14.1 Glossary

- **Active Session:** A shopping trip currently in progress
- **Cart:** Temporary storage for scanned products during a session
- **Checkout:** The process of completing a shopping trip and saving it to history
- **Guest User:** A user using the app without creating an account
- **Scan:** The act of capturing a product's barcode and details
- **Session:** A single shopping trip from start to checkout

- **Spending Limit:** The budget set by the user for a shopping session

---

14.2 References

- [React Native Documentation](https://reactnative.dev/)
- [Expo Documentation](https://docs.expo.dev/)
- [Supabase Documentation](https://supabase.com/docs)
- [Expo Camera API](https://docs.expo.dev/versions/latest/sdk/camera/)
- [Expo Barcode Scanner](https://docs.expo.dev/versions/latest/sdk/bar-code-scanner/)

14.3 Change Log

Version	Date	Author	Changes
1.0	Nov 3, 2025	Product Team	Initial PRD creation

---

15. Sign-Off

This Product Requirements Document requires approval from:

- [ ] **Product Manager:** _____
- [ ] **Engineering Lead:** _____
- [ ] **Design Lead:** _____
- [ ] **QA Lead:** _____
- [ ] **Stakeholder:** _____

Date: _____

Document Status: ✅ Ready for Review
Next Steps:
1. Stakeholder review and feedback
2. Technical feasibility assessment
3. Design mockups creation
4. Development sprint planning
5. Kickoff meeting

End of Product Requirements Document

Total Pages: 30+
Last Updated: November 3, 2025
Version: 1.0 DRAFT

This PRD provides a comprehensive blueprint for building the Simple Grocery Tracker app from concept to launch. All sections are designed to be actionable by development, design, and product teams.